

TRABAJO PRÁCTICO: BÚSQUEDA BINARIA

Asignatura: Programación III

Unidad: Algoritmos de Búsqueda (Divide y Conquista)

🎯 Objetivos de Aprendizaje

- Comprender e implementar el algoritmo de **Búsqueda Binaria** utilizando el enfoque de **Divide y Conquista** (recursivo).
- Aplicar el paso de parámetros adecuado mediante índices (`inicio` y `fin`) para garantizar eficiencia en memoria.
- Evaluar empíricamente la complejidad temporal $\Theta(\log n)$ frente a una búsqueda lineal $\Theta(n)$.

📋 Contexto y Enunciado del Ejercicio

La empresa de logística **ExpressDelivery** necesita optimizar el sistema de consulta de su base de datos de envíos. Poseen un registro masivo de paquetes identificados por un **código de seguimiento entero**, el cual se almacena de forma estrictamente **ordenada de manera creciente**.

Se solicita implementar un módulo en Java que contenga el método de búsqueda binaria mediante Divide y Conquista y realice una prueba comparativa de rendimiento frente a un enfoque lineal.

💻 Requerimientos Prácticos

Parte 1: Implementación del Algoritmo Recursivo

Crear la clase `BusquedaBinaria` con el siguiente método estático:

```
public static boolean buscar(int[] S, int x, int inicio, int fin)
```

Especificaciones técnicas del método:

- **Parámetros:** Arreglo ordenado `S`, elemento a buscar `x`, e índices de control `inicio` y `fin`.
- **Retorno:** `true` si $x \in S$, o `false` en caso contrario.
- **Estructura Divide y Conquista:**
 - **Caso base 1:** Si la subsecuencia no es válida (`inicio > fin`), retornar `false`.
 - **Caso base 2:** Calcular la mitad `mitad = inicio + (fin - inicio) / 2`. Si `S[mitad] == x`, retornar `true`.
 - **Subproblemas:** Si $x > S[mitad]$, buscar recursivamente en el subsegmento derecho; si $x < S[mitad]$, buscar en el subsegmento izquierdo.

Parte 2: Análisis de Rendimiento y Comparativa

En una clase `Main`, implementar la siguiente prueba de estrés:

1. Generar un arreglo `S` de tamaño $N = 10.000.000$ de enteros ordenados consecutivos (ejemplo: números pares `0, 2, 4, 6, ...`).

2. Realizar la búsqueda del número $x = -1$ (peor caso: el elemento no existe) utilizando:
 - Búsqueda Lineal $\Theta(n)$.
 - Búsqueda Binaria Recursiva $\Theta(\log n)$.
3. Medir y mostrar en milisegundos el tiempo de ejecución de cada algoritmo utilizando `System.nanoTime()`.

? Preguntas para el Informe / Cierre

Responde a las siguientes preguntas teóricas al finalizar la práctica:

1. **Eficiencia espacial vs. temporal:** ¿Por qué en la implementación en Java es indispensable trabajar con índices (`inicio` y `fin`) sobre el mismo arreglo en lugar de pasar copias de subarreglos (`Arrays.copyOfRange`)?
2. **Ecuación de Recurrencia:** Sabiendo que la ecuación de recurrencia es $T(n) = T(n/2) + c$, ¿cuántas comparaciones como máximo realiza la búsqueda binaria si el arreglo tiene **1.000.000** de elementos?
3. **Casos Límites:** ¿Qué ocurre si la búsqueda binaria se ejecuta sobre un arreglo que no está ordenado? ¿Falla la compilación, arroja excepción o genera un resultado erróneo? Justifica.